

Assignments

T0 – Basic Setup

Notice

The information in this document is subject to change without notice, and should not be construed as a commitment by HENSOLDT Cyber.

HENSOLDT Cyber provides no warranty, express or implied, including warranties of merchantability and fitness for a particular purpose for the information contained in this document, and assumes no responsibility for any errors that may appear in this document. In no event shall HENSOLDT Cyber or any of its suppliers be liable for direct, indirect, special, incidental or consequential damages of any nature or kind arising from the use of this document, or from the use of any hardware or software described in this document, even if HENSOLDT Cyber or its suppliers have been advised of the possibility of such damages.

This document and parts hereof must not be reproduced or copied without written permission from HENSOLDT Cyber and the contents hereof must not be imparted to a third party nor used for any unauthorized purpose. All rights to registrations and trademarks reside with their respective owners.

Copyright © 2021 HENSOLDT Cyber GmbH – All rights reserved.

Trademarks

TRENTOS® is a trademark of HENSOLDT Cyber GmbH.

sel4® is a trademark of LF Projects, LLC.

Table of Contents

Table of Contents	3
1 General	4
2 Components	5
3 Workflow	6
4 Setup	7
4.1 Design the components	7
4.1.1 Re-usable components	7
4.1.2 The TestApp component	11
4.2 Provision of a global configuration	13
4.3 Link the components together	15
4.4 Fill the components with life	18
4.4.1 RamDisk and StorageServer	18
4.4.2 The TestApp component	18
4.5 Define the build process	24
5 Building the demo	26
6 Running the demo	27

1 General

The basic setup shall showcase the usage of the TRENTOS-M storage facilities in combination with the TRENTOS-M FileSystem API. Therefore, a memory-backed storage backend shall be used to provide two partitions which can be utilized by an application on top.

The basic setup shall then provide a simple use case, where:

- the memory-backed storage backend is connected to the OS via a storage driver to provide basic persistent storage
- the storage backend is split into two partitions with the help of some kind of partition manager
- the partition(s) are used to initialize two FAT file systems, which allow basic file operations (e.g. read, write) by utilizing the TRENTOS-M FileSystem API

2 Components

The following TRENTOS-M components can be re-used for the setup:

- RamDisk: provides the memory-backed storage backend
- StorageServer: offers functionality for the partitioning of a storage backend

In addition to the TRENTOS-M components mentioned above, a test application (TestApp) has to be created. The TestApp component uses the file system library from TRENTOS-M on top of the StorageServer, in order to provide file storage facilities. It therefore utilizes the TRENTOS-M FileSystem API via the `if_OS_Storage` interface.

3 Workflow

The figure below describes the general interplay between different TRENTOS-M components. For our basic setup, let's highlight the persistent storage use case:

- An application uses the file system library to store files persistently. The FAT and SPIFFS file system types are offered.
- To abstract from the underlying disk, the file system makes use of the StorageServer, which offers two partitions.
- In order to communicate with the disk, a specific driver (RamDisk) is required.

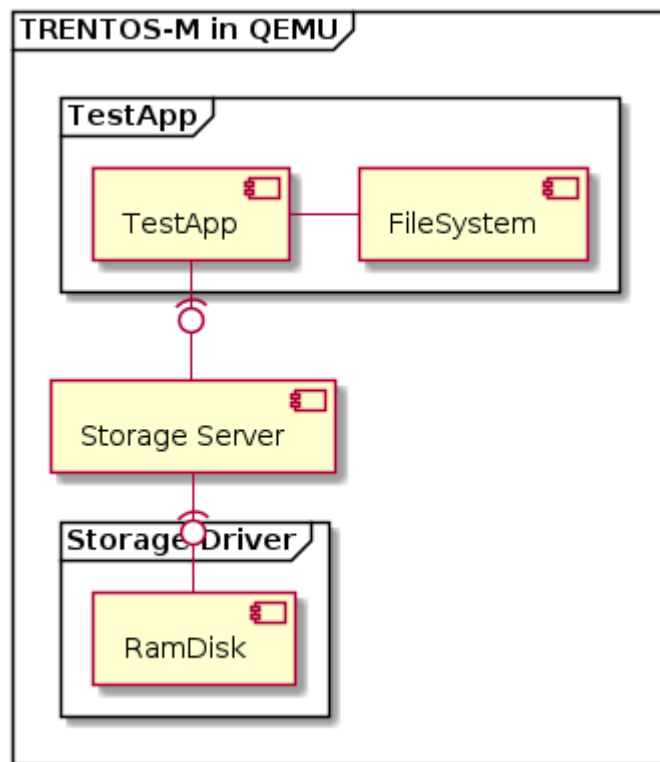


Figure 1: Demo Training Course - CAMkES Architecture

4 Setup

First of all, a new project has to be created, which shall comprise all the logic required. Therefore, a new folder has to be created in `/sdk/demos/`, which is called `demo_training_course`. To comply with the SDK structure, an empty folder `src` has to be created additionally, which is referred to as root folder throughout the tutorial.

4.1 Design the components

In order to start with development, an additional folder `components` has to be created within `/sdk/demos/demo_training_course/src/`, which holds the CAMkES component descriptions. Each TRENTOS-M application consists of at least one CAMkES component. A CAMkES component and its features are defined in form of a `.camkes` file.

Within the basic setup project, basically every component makes use of interfaces, in order to interact with the environment. An interface is also defined in form of a `.camkes` file. Using TRENTOS-M, interfaces can thereby either be used based on the TRENTOS-M API or defined directly by its own. Within the scope of the basic setup, the former case will be used and described in the following.

4.1.1 Re-usable components

As already stated above, the following TRENTOS-M components can be re-used:

- RamDisk
- StorageServer

4.1.1.1 RamDisk

The RamDisk component can be found in `/sdk/components/RamDisk` and is defined as following:

```
/* Copyright (C) 2020, HENSOLDT Cyber GmbH */

/**
 * @file
 * @brief   Driver for the Volatile Memory storage (RamDisk)
 *
 * This component acts like a disk driver (secondary storage), but in reality
 * stores the data in the volatile memory are (RAM).
 *
 * It shall be used, when relatively fast storage is needed and it is ok for the
 * data to be lost between reset cycles. Use cases for for example caching or
 * testing.
 *
 * @note    The underlying memory is volatile, so expected loss of data between
 *          reboots!
 *
 * @note    The size of the storage is configured with the macro
```

Setup

```
*          RAMDISK_SIZE_BYTES, please make sure that it is defined in the main
*          config header.
*
* @see @link https://en.wikipedia.org/wiki/RAM_drive @endlink
*/

#pragma once

/** @cond SKIP_IMPORTS */
import <std_connector.camkes>;
import <if_OS_Storage.camkes>;
/** @endcond */

//-----

#define RamDisk_COMPONENT_DEFINE( \
    _name_) \
    \
    component _name_ { \
        provides    if_OS_Storage    storage_rpc; \
        dataport    Buf              storage_port; \
    }

...
```

Code Block 1: RamDisk.camkes

For now, only the `RamDisk_COMPONENT_DEFINE` macro is of importance. The RamDisk thereby

- implements the TRENTOS-M storage interface `if_OS_Storage`, naming it `storage_rpc` and offering functionality to other components (keyword `provides`)
- implements a `dataport` (`storage_port`), which represents CAMkES's abstraction of shared memory, and allows for shared memory based communication between components. The default dataport type is `Buf`, which is implemented in form of a byte array.

4.1.1.2 StorageServer

The StorageServer component can be found in `/sdk/components/StorageServer` and is defined as following:

```
/*
 * Copyright (C) 2020, Hensoldt Cyber GmbH
 */

/*
 * StorageServer CAMkES Component
 *
 * to be used as:
 *
 *      #include "StorageServer/camkes/StorageServer.camkes"
 *
 *      StorageServer_COMPONENT_DEFINE(StorageServer_1)
 *      StorageServer_COMPONENT_DEFINE(StorageServer_2)
 *
 */

#pragma once

import <std_connector.camkes>;
import <if_OS_Storage.camkes>;

#include "lib_macros/ForEach.h"

//-----

struct StorageServer_ClientConfig {
    int64_t size;
    int64_t offset;
}

struct StorageServer_Config {
    StorageServer_ClientConfig clients[];
}

/*
 * Declare the StorageServer with a given type name:
 *
 *      StorageServer_COMPONENT_DEFINE(
```

Setup

```

*           <name>
*       )
*/
#define StorageServer_COMPONENT_DEFINE( \
    _name_) \
    \
    component _name_ { \
        uses          if_OS_Storage    storage_rpc; \
        dataport      Buf              storage_port; \
        provides      if_OS_Storage    storageServer_rpc; \
        maybe dataport Buf              storageServer1_port; \
        maybe dataport Buf              storageServer2_port; \
        maybe dataport Buf              storageServer3_port; \
        maybe dataport Buf              storageServer4_port; \
        maybe dataport Buf              storageServer5_port; \
        maybe dataport Buf              storageServer6_port; \
        maybe dataport Buf              storageServer7_port; \
        maybe dataport Buf              storageServer8_port; \
        attribute      StorageServer_Config    storageServer_config; \
    }

...

```

Code Block 2: StorageServer.camkes

For now, only the `StorageServer_COMPONENT_DEFINE` macro is of importance. The `StorageServer` thereby also interacts with the TRENTO-S-M storage interface `if_OS_Storage`. Two different ways are provided:

- invocation of functions from the TRENTO-S-M storage interface `if_OS_Storage`, naming it `storage_rpc`, using functionality that is provided by other components (keyword `uses`). These functions are implemented by an underlying storage abstraction, e.g. the aforementioned `RamDisk` component.
- implementation of the TRENTO-S-M storage interface `if_OS_Storage`, naming it `storageServer_rpc`, and offering functionality to other components (keyword `provides`)

In addition, the component implements up to 9 dataports. The exact amount is depending on the amount of connected client components. Currently, a maximum of 8 parallel clients is allowed. One dataport is used for communication with the underlying storage (`storage_port`), the others (e.g. `storageServer1_port`) are used for communication to components running on top (e.g. our `TestApp`).

4.1.1.3 Excursus: Workflow

In order to make use of the presented components, in general the following steps have to be taken:

1. Both components first have to be declared in the project's `CMakeLists.txt` file.
2. The size of the RamDisk must be specified in an additional configuration file. In our setup the file is called `system_config.h`.
3. Afterwards, both components have to be declared in the application's main `.camkes` file. In our setup the file is called `DemoTrainingCourse.camkes`.
4. Within `DemoTrainingCourse.camkes`, the components then must be defined:
 - a. The RamDisk via the `RamDisk_COMPONENT_DEFINE` macro.
 - b. The StorageServer via the `StorageServer_COMPONENT_DEFINE` macro.
5. Afterwards, the components must be instantiated.
6. The RamDisk instance is then connected to the StorageServer instance via the `StorageServer_INSTANCE_CONNECT` macro.
7. Via the `StorageServer_INSTANCE_CONNECT_CLIENTS` macro, the TestApp will be connected later.

The workflow will be depicted in more detail throughout the tutorial.

4.1.2 The TestApp component

Within the folder `components`, we will create a new folder `TestApp`. Inside this folder, we will place the TestApp component description in form of the `TestApp.camkes` file.

```
/**
 * Copyright (C) 2020, Hensoldt Cyber GmbH
 */

import <if_OS_Storage.camkes>;

#include "system_config.h"

component TestApp {
    control;

    // For underlying storage - partition 1
    uses      if_OS_Storage      storage_rpc_1;
    dataport  Buf                storage_dp_1;

    // For underlying storage - partition 2
    uses      if_OS_Storage      storage_rpc_2;
    dataport  Buf                storage_dp_2;
}
```

Code Block 3: TestApp.camkes

Setup

We define a component `TestApp` by using the CAMkES type `component`. The CAMkES keyword `control` is used here to reflect an active component, meaning that this component will contain the main function (within CAMkES prototyped as `run`) which is executed by a dedicated thread of control.

The `TestApp` component shall use the `StorageServer` in order to abstract from the underlying physical disk and provide access to two logical partitions. In order to allow for communication between the component and the underlying `StorageServer`, the component has to be equipped with an RPC interface and a `dataport` interface for each of the two partitions provided. As an interface between the `TestApp` and the `StorageServer` again the TRENTOS-M standard interface `if_OS_Storage` is used.

As already shown above, the `StorageServer` component implements the TRENTOS-M standard procedure interface `if_OS_Storage` via `provides`, in order to provide functions that can be called by the `TestApp` component (keyword `uses`). As the `if_OS_Storage` interface definition is provided by TRENTOS-M (here: `if_OS_Storage.camkes` in `/sdk/os_core_api/camkes`), it has to be included in the `TestApp.camkes` file accordingly with the help of the `import` statement.

The `TestApp` component is therefore able to use the functions

- `write()`
- `read()`
- `erase()`
- `getSize()`
- `getBlockSize()`
- `getState()`

which are defined by the `if_OS_Storage` interface in the file `if_OS_Storage.camkes`. The respective function logic is implemented by the `StorageServer` component.

4.2 Provision of a global configuration

After having defined the required components to be used in our application, we need to specify a global configuration, containing parameters that are required for the seamless interaction of our components. Therefore, we will create a respective config file called `system_config.h`, which resides in the project's root folder. The content of the file looks like the following:

```
/**
 * Copyright (C) 2019, Hensoldt Cyber GmbH
 *
 * OS libraries configurations
 */

#pragma once

//-----
// Debug
//-----

#ifndef NDEBUG
#   define Debug_Config_STANDARD_ASSERT
#   define Debug_Config_ASSERT_SELF_PTR
#else
#   define Debug_Config_DISABLE_ASSERT
#   define Debug_Config_NO_ASSERT_SELF_PTR
#endif

#define Debug_Config_LOG_LEVEL           Debug_LOG_LEVEL_INFO
#define Debug_Config_INCLUDE_LEVEL_IN_MSG
#define Debug_Config_LOG_WITH_FILE_LINE

//-----
// Memory
//-----

#define Memory_Config_USE_STDLIB_ALLOC

//-----
// RamDisk size
//-----

#define RAMDISK_SIZE_BYTES    (2 * 1024 * 1024)

//-----
```

Setup

```
// StorageServer
//-----
#define FILESYSTEM_1_STORAGE_OFFSET      0
#define FILESYSTEM_1_STORAGE_SIZE        (1024*1024)
#define FILESYSTEM_2_STORAGE_OFFSET      (1024*1024)
#define FILESYSTEM_2_STORAGE_SIZE        (1024*1024)
```

Code Block 4: system_config.h

Basically, the content of the config file strongly depends on the respective use case and the individual TRENTOS-M libraries used. In our example, we basically make use of `RamDisk` and `StorageServer`, so the configuration of these parts is required. The required configuration parameters are each defined by the respective library. In order to identify them, one has to have a look at the appropriate library source files. The sections *Debug* and *Memory* contain rather general information. Of main interest in our setup are the configurations of the `RamDisk` and the `StorageServer`. For the `RamDisk` we have to provide the respective disk size (here: 2 MiB). For the `StorageServer` we have to specify the available partitions. Each partition contains an offset and a size (here splitting the `RamDisk` in equal halves).

4.3 Link the components together

After having described and defined the individual CAMkES components, they have to be instantiated. In addition, also their individual interconnection and interplay has to be specified. Therefore, we will create an additional .camkes file called DemoTrainingCourse.camkes, which resides in the project's root folder.

```
/**
 * Main CAMkES configuration file of the Training Course demo application.
 *
 * Copyright (C) 2020, Hensoldt Cyber GmbH
 */

import <std_connector.camkes>;

#include "system_config.h"

//-----
// Storage
//-----

#include "RamDisk/RamDisk.camkes"
RamDisk_COMPONENT_DEFINE(RamDisk)

#include "StorageServer/camkes/StorageServer.camkes"
StorageServer_COMPONENT_DEFINE(StorageServer)

import "components/TestApp/TestApp.camkes";

//-----
// System
//-----

assembly {
    composition {
        //-----
        // Storage
        //-----
        component    RamDisk            ramDisk;

        //-----
        // StorageServer
        //-----
        component    StorageServer      storageServer;

        StorageServer_INSTANCE_CONNECT(
```

Setup

```

        storageServer,
        ramDisk.storage_rpc, ramDisk.storage_port
    )

    StorageServer_INSTANCE_CONNECT_CLIENTS (
        storageServer,
        testApp.storage_rpc_1, testApp.storage_dp_1,
        testApp.storage_rpc_2, testApp.storage_dp_2
    )

    //-----
    // Demo Component (using File System)
    //-----
    component    TestApp    testApp;
}

configuration {

    StorageServer_INSTANCE_CONFIGURE_CLIENTS (
        storageServer,
        FILESYSTEM_1_STORAGE_OFFSET, FILESYSTEM_1_STORAGE_SIZE,
        FILESYSTEM_2_STORAGE_OFFSET, FILESYSTEM_2_STORAGE_SIZE
    )

    StorageServer_CLIENT_ASSIGN_BADGES (
        testApp.storage_rpc_1,
        testApp.storage_rpc_2
    )

}
}

```

Code Block 5: DemoTrainingCourse.camkes

In order to make use of the defined components, we have to import

- `std_connector.camkes`, which according to the CAMkES manual is a built-in file that defines the standard CAMkES connector types
- the respective `.camkes` file of the `TestApp` component

and to include

- `system_config.h`, which contains the global configuration
- the respective `.camkes` files of the `RamDisk` and the `StorageServer` components

As already listed in the workflow description above, both the `RamDisk` and the `StorageServer` components then must be defined:

Setup

1. The `RamDisk` via the `RamDisk_COMPONENT_DEFINE` macro.
2. The `StorageServer` via the `StorageServer_COMPONENT_DEFINE` macro.

All the components and their connections are then placed inside a top-level element called `assembly`, which defines the complete description of our system. The system thereby consists of two building blocks:

- a `composition`, which describes the system, i.e. all component and connector instantiations and
- an (optional) `configuration`, which contains system parameter settings.

Within the `composition`, we create instances of the `RamDisk` (`ramDisk`), the `StorageServer` (`storageServer`) and the `TestApp` (`testApp`). The `RamDisk` instance is then connected to the `StorageServer` instance via the `StorageServer_INSTANCE_CONNECT` macro. Via the `StorageServer_INSTANCE_CONNECT_CLIENTS` macro, the `TestApp` is connected as a client to the `StorageServer` as well.

Within the `configuration`, we specify the respective address ranges of the two partitions required by `TestApp`, which are mapped based on their sizes and offsets via the macro `StorageServer_INSTANCE_CONFIGURE_CLIENTS`. The respective values can be found in the `system_config.h` file. By using the macro `StorageServer_CLIENT_ASSIGN_BADGES`, we also assign badges to the two `StorageServer` clients.

4.4 Fill the components with life

In order to provide a fully working application, the application skeleton created with the help of CAMkES has to be filled with respective application logic. Apart from the TRENTOS-M components `RamDisk` and `StorageServer` (which are already working out of the box), each CAMkES component has to be extended by utilizing the C programming language.

4.4.1 `RamDisk` and `StorageServer`

Both the `RamDisk` and the `StorageServer` are already implemented as they are TRENTOS-M standard components. The respective component code can be found in

- `sdk/components/RamDisk`
- `sdk/components/StorageServer`

4.4.2 The `TestApp` component

Within the `TestApp` folder we create a new folder `src`. Here, we create a file called `TestApp.c`, which shall contain the `TestApp` component logic. We will now go through the `TestApp.c` implementation step by step, demonstrating the workflow of the demo and therefore highlighting the relevant parts within separated sections. `TestApp.c` represents the `TestApp` component. Note: the file name `TestApp` thereby maps to the name of the corresponding component specified in the `TestApp.camkes` file.

First of all, we have to include several files:

- `OS_FileSystem.h` for being able to access the TRENTOS-M FileSystem API
- `lib_debug/Debug.h` for using the TRENTOS-M debugging
- `string.h` for using the C String Library and respective functions (e.g. `memcmp` function)

We furthermore have to include `camkes.h`, which is generated by the CAMkES build system, to gain access to the attributes of our CAMkES component descriptions, e.g. the dataports `storage_dp_1` and `storage_dp_2` and the RPC interfaces `storage_rpc_1` and `storage_rpc_2` defined in `TestApp.camkes`, for being able to exchange data between the `TestApp` and the `StorageServer`.

We choose the FAT file system for both partition 1 and partition 2 and provide a respective configuration (`fatCfg_1` and `fatCfg_2`).

Setup

```
/*
 * Copyright (C) 2020, Hensoldt Cyber GmbH
 */

#include "OS_FileSystem.h"

#include "lib_debug/Debug.h"
#include <string.h>

#include <camkes.h>

//-----
static OS_FileSystem_Config_t fatCfg_1 =
{
    .type = OS_FileSystem_Type_FATFS,
    .size = OS_FileSystem_USE_STORAGE_MAX,
    .storage = IF_OS_STORAGE_ASSIGN(
        storage_rpc_1,
        storage_dp_1),
};

static OS_FileSystem_Config_t fatCfg_2 =
{
    .type = OS_FileSystem_Type_FATFS,
    .size = OS_FileSystem_USE_STORAGE_MAX,
    .storage = IF_OS_STORAGE_ASSIGN(
        storage_rpc_2,
        storage_dp_2),
};
```

Code Block 6: TestApp.c

We then provide a function `test_OS_FileSystem()`, which we will use for exemplarily showcasing the most relevant file system functionality. For file system related operations we follow the reference workflow provided in the TRENTOS-M Handbook, chapter "TRENTOS-M FileSystem API". In the beginning, we define the required variables and structures like a file system handle (`hFs`) and a file handle (`hFile`), the file name, the file size and the actual file data to be written on the disk.

Setup

```
//-----  
static void  
test_OS_FileSystem(OS_FileSystem_Config_t* cfg)  
{  
    OS_Error_t ret;  
    OS_FileSystem_Handle_t hFs;  
  
    static const char* fileName = "testfile.txt";  
    static const uint8_t fileData[] =  
    {  
        0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77  
    };  
    static const off_t fileSize = sizeof(fileData);  
    static OS_FileSystemFile_Handle_t hFile;
```

Code Block 7: TestApp.c

We then initialize the file system on the respective partition provided by one of the file system configurations specified before (OS_FileSystem_Config_t* cfg).

```
// Init file system  
if ((ret = OS_FileSystem_init(&hFs, cfg)) != OS_SUCCESS)  
{  
    Debug_LOG_ERROR("OS_FileSystem_init() failed, code %d", ret);  
}
```

Code Block 8: TestApp.c

Afterwards, we format and mount the file system.

```
// Format file system  
if ((ret = OS_FileSystem_format(hFs)) != OS_SUCCESS)  
{  
    Debug_LOG_ERROR("OS_FileSystem_format() failed, code %d", ret);  
}  
  
// Mount file system  
if ((ret = OS_FileSystem_mount(hFs)) != OS_SUCCESS)  
{  
    Debug_LOG_ERROR("OS_FileSystem_mount() failed, code %d", ret);  
}
```

Code Block 9: TestApp.c

Setup

Now, everything is prepared for actually creating new files. In our case, we want to open the file with the provided `fileName` in case it is already existing or otherwise create a new file (`OS_FileSystem_OpenFlags_CREATE`). We also open the file in read/write mode, as otherwise we won't be able to do any file-related operations.

```
// Open file
if((ret = OS_FileSystemFile_open(hFs, &hFile, fileName,
                                OS_FileSystem_OpenMode_RDWR,
                                OS_FileSystem_OpenFlags_CREATE)) != OS_SUCCESS)
{
    Debug_LOG_ERROR("OS_FileSystemFile_open() failed, code %d", ret);
}
```

Code Block 10: TestApp.c

By using the respective file handle (`hFile`), we can now write the content to the newly created file.

```
// Write to the file
off_t to_write, written;
to_write = fileSize;
written = 0;

while (to_write > 0)
{
    if((ret = OS_FileSystemFile_write(hFs, hFile, written, sizeof(fileData),
fileData)) != OS_SUCCESS)
    {
        Debug_LOG_ERROR("OS_FileSystemFile_write() failed, code %d", ret);
    }

    written += sizeof(fileData);
    to_write -= sizeof(fileData);
}
```

Code Block 11: TestApp.c

Afterwards, we want to test if all data was written correctly by reading back the file from the disk via the `OS_FileSystemFile_read()` function and comparing the content with the help of the `memcmp()` function.

Setup

```
// Read from the file
uint8_t buf[sizeof(fileData)];
off_t to_read, read;
to_read = fileSize;
read = 0;

while (to_read > 0)
{
    if((ret = OS_FileSystemFile_read(hFs, hFile, read, sizeof(buf), buf)) !=
OS_SUCCESS)
    {
        Debug_LOG_ERROR("OS_FileSystemFile_read() failed, code %d", ret);
    }

    if(memcmp(fileData, buf, sizeof(buf)))
        Debug_LOG_ERROR("File content read does not equal file content to be
written.");

    read    += sizeof(buf);
    to_read -= sizeof(buf);
}
```

Code Block 12: TestApp.c

If everything went fine, we close the file again and clean up by unmounting and freeing the file system.

```
// Close file
if((ret = OS_FileSystemFile_close(hFs, hFile)) != OS_SUCCESS)
{
    Debug_LOG_ERROR("OS_FileSystemFile_close() failed, code %d", ret);
}

// Clean up
if((ret = OS_FileSystem_unmount(hFs)) != OS_SUCCESS)
{
    Debug_LOG_ERROR("OS_FileSystem_unmount() failed, code %d", ret);
}

if ((ret = OS_FileSystem_free(hFs)) != OS_SUCCESS)
{
    Debug_LOG_ERROR("OS_FileSystem_free() failed, code %d", ret);
}
}
```

Code Block 13: TestApp.c

Setup

According to the CAMkES manual, our component also has to implement a `run()` function, as we used the `control` keyword in the `TestApp.camkes` file. `TestApp` is as an active component, which means the main function is required and an active thread of control is created. The CAMkES manual expects the provision of a function `int run(void)` (which is prototyping the main function) that is invoked after component initialization is complete. The `run()` function marks the entry point of a CAMkES component. Within the `run()` function we use our `test_OS_FileSystem()` function for creating a file on each of the two partitions by providing the respective file system configurations `fatCfg_1` and `fatCfg_2`.

```
//-----  
int run()  
{  
    // Work on file system 1 (residing on partition 1)  
    test_OS_FileSystem(&fatCfg_1);  
  
    // Work on file system 2 (residing on partition 2)  
    test_OS_FileSystem(&fatCfg_2);  
  
    Debug_LOG_INFO("Demo completed successfully.");  
  
    return 0;  
}
```

Code Block 14: TestApp.c

4.5 Define the build process

As stated in the TRENTOS-M Handbook, chapter "TRENTOS-M Buildsystem", the TRENTOS-M SDK makes use of the CMake build system. We therefore have to define a corresponding `CMakeLists.txt` file, which resides in the project's root folder. The CMake build file thereby uses both general CMake commands as well as CAmkES-specific CMake commands. Some macros were also added by TRENTOS-M. Let's have a look at the overall structure.

```
#
# Demo for the Training Course
#
# Copyright (C) 2020, Hensoldt Cyber GmbH
#

cmake_minimum_required(VERSION 3.7.2)

os_set_config_file(system_config.h)

project(system_config C)
add_library(${PROJECT_NAME} INTERFACE)
target_include_directories(${PROJECT_NAME} INTERFACE ${CMAKE_CURRENT_LIST_DIR})

#-----
project(demo_training_course C)

RamDisk_DeclareCAmkESComponent (
    RamDisk
    system_config
)

StorageServer_DeclareCAmkESComponent (
    StorageServer
)

DeclareCAmkESComponent (
    TestApp
    SOURCES
        components/TestApp/src/TestApp.c
    C_FLAGS
        -Wall
        -Werror
    LIBS
        system_config
        lib_debug
        os_core_api
        os_filesystem
)
```


Setup

```
)  
  
DeclareAndCreateCamkESSystem(DemoTrainingCourse.camkes)  
GenerateSimulateScript()
```

Code Block 15: CMakeLists.txt

We first create a separate project for our global `system_config.h` file, to be able to include and add it to our components later. We therefore make use of the TRENTOS-M macro `os_set_config_file`, which is defined in `/sdk/CMakeLists.txt`. More information on the TRENTOS-M specific macros can be found in the TRENTOS-M Handbook, chapter "TRENTOS-M Buildsystem". We then declare our main project `demo_training_course`.

Afterwards, we have to specify the setup for our three CAMkES components: `RamDisk`, `StorageServer` and `TestApp`. The two TRENTOS-M components shipped with the SDK provide a dedicated and simple CAMkES macro each, which hides most of the underlying configuration and dependencies:

- `RamDisk_DeclareCamkESComponent`
- `StorageServer_DeclareCamkESComponent`

Each macro requires the name of the component (e.g. `RamDisk`, `StorageServer`). `RamDisk` additionally requires the `system_config` project, to allow for accessing data from the global configuration within CAMkES and C code. For our own component `TestApp` we use the standard CAMkES command `DeclareCamkESComponent`. Besides the name, the command also takes the following parameters:

- **SOURCES:** here we state the path to a component's respective implementation files, which are written in the C programming language.
- **C_FLAGS:** Additional compiler flags.
- **LIBS:** According to the CAMkES manual, this field can be used to specify any argument that can be ordinarily given to CMake's `target_link_libraries()` macro. Here, we provide the TRENTOS-M libraries that are used within the respective component, e.g. the `os_core_api`.

In the end, we have to use several CAMkES-specific CMake commands:

- Define the root application, and therefore provide our `DemoTrainingCourse.camkes` file to the `DeclareAndCreateCamkESSystem()` command. This command internally uses the standard CAMkES commands `DeclareCamkESRootserver()` and `GenerateCamkESRootServer()` (defined in `/sdk/sdk-sel4-camkes/tools/camkes/camkes.cmake`), for generating the CAMkES root server and the global configuration.
- Generate a `./simulate` style script by calling `GenerateSimulateScript()` (defined in `/sdk/sdk-sel4-camkes/tools/sel4/cmake-tool/helpers/simulation.cmake`)

5 Building the demo

To build our demo, from within the SDK root folder we start the TRENTOS-M build container with the following parameters:

```
sdk/scripts/open_trentos_build_env.sh sdk/build-system.sh  
sdk/demos/demo_training_course/src zynq7000 build-zynq7000-Debug-demo_training_course -  
DCMAKE_BUILD_TYPE=Debug
```

Code Block 16: Call build.sh in the container environment

As a result, the folder "**build-zynq7000-Debug-demo_training_course**" is created, containing all the build artefacts. Please also refer to the "Getting Started" guide for more helpful information on how to build TRENTOS-M systems.

6 Running the demo

In order to run the demo in QEMU, we switch to the build directory of the demo (build-zynq7000-Debug-demo_training_course) and start the test docker container from within (see "Getting Started" guide for more information on how to test a TRENTO-M system) with the help of the `simulate` run script.

```
cd build-zynq7000-Debug-demo_training_course
../sdk/scripts/open_trentos_test_env.sh ./simulate
```

Code Block 17: Call `run_demo.sh`

QEMU will then provide the following output:

```
ELF-loader started on CPU: ARM Ltd. Cortex-A9 r3p0
  paddr=[717000..adeeb3]
No DTB passed in from boot loader.
Looking for DTB in CPIO archive...found at 8260bc.
Loaded DTB from 8260bc.
  paddr=[3c000..3efff]
ELF-loading image 'kernel'
  paddr=[0..3bfff]
  vaddr=[e0000000..e003bfff]
  virt_entry=e0000000
ELF-loading image 'capdl-loader'
  paddr=[3f000..317fff]
  vaddr=[10000..2e8fff]
  virt_entry=18824
Enabling MMU and paging
Bootstrapping kernel
Warning: Could not infer GIC interrupt target ID, assuming 0.
Booting all finished, dropped to user space
main@main.c:2114 Starting CapDL Loader...
main@main.c:2116 CapDL Loader done, suspending...
  INFO: /host/sdk/components/RamDisk/RamDisk.c:278: RamDisk has size of 2097152 bytes
  INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:504: storage
medium size: 2097152 bytes
  INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:522: client 1:
offset=0, size=1048576
  INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:522: client 2:
offset=1048576, size=1048576
  INFO: /host/sdk/libs/os_filesystem/src/OS_FileSystem.c:125: Maximizing file system
according to size reported by the storage layer (1048576 bytes)
  INFO: /host/sdk/libs/os_filesystem/src/lib/FatFs.c:197: Using FATFS (sector_count =
2048, sector_size = 512, block_size = 16)
```

Running the demo

```
INFO: /host/sdk/libs/os_filesystem/src/OS_FileSystem.c:125: Maximizing file system
according to size reported by the storage layer (1048576 bytes)
INFO: /host/sdk/libs/os_filesystem/src/lib/FatFs.c:197: Using FATFS (sector_count =
2048, sector_size = 512, block_size = 16)
INFO:
/host/sdk/demos/demo_training_course_ss2021/src/components/TestApp/src/TestApp.c:135:
Demo completed successfully.
```

Code Block 18: QEMU Terminal

Congratulations, you have successfully built your first own TRENTOS-M application.



HENSOLDT Cyber GmbH

Willy-Messerschmitt-Straße 3
82024 Taufkirchen,
Deutschland

www.hensoldt-cyber.com